TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP.HCM
VIỆN CÔNG NGHỆ THÔNG TIN, ĐIỆN, ĐIỆN TỬ

Chương 2:
Các thuật toán xử lý ảnh
(phần 1)

TS. Nguyễn Thị Khánh Tiên
tienntk@ut.edu.vn

Tổng quan về các thuật toán xử lý ảnh

Xử lý ảnh là một lĩnh vực rộng lớn trong khoa học máy tính, tập trung vào việc thao tác và phân tích hình ảnh kỹ thuật số. Các thuật toán được phát triển để thực hiện các tác vụ khác nhau, từ những thao tác đơn giản như điều chỉnh độ sáng đến những nhiệm vụ phức tạp như nhận diện khuôn mặt. Dưới đây là một số nhóm thuật toán xử lý ảnh phổ biến:

1. Các thuật toán cơ bản:

- Đọc và hiển thị ảnh
- Chuyển đổi không gian màu
- Cắt xén và thay đổi kích thước
- Điều chỉnh độ sáng, độ tương phản

2. Lọc ảnh:

- Lọc trung bình: Làm mờ hình ảnh để giảm nhiễu.
- Lọc Gaussian: Làm mờ hình ảnh một cách trơn tru hơn.
- Lọc median: Loại bỏ các điểm nhiễu đơn lẻ.
- Lọc bilateral: Kết hợp giữa lọc không gian và lọc miền cường độ.

3. Phát hiện cạnh:

- Toán tử Sobel: Phát hiện các cạnh theo cả phương ngang và dọc.
- Toán tử Canny: Phát hiện cạnh với độ chính xác cao.
- Laplacian: Phát hiện các cạnh dựa trên độ thay đổi đột ngột của cường độ pixel.

4. Phân đoạn ảnh:

- Phân ngưỡng: Chia hình ảnh thành vùng dựa trên một ngưỡng cường độ.
- K-means: Phân cụm các pixel thành K nhóm.
- Otsu's thresholding: Tự động tìm ngưỡng tốt nhất để phân chia ảnh thành hai vùng.
- Region-based segmentation: Phân đoạn dựa trên các vùng liên thông có cùng tính chất.

5. Biến đổi hình học:

- Phóng to, thu nhỏ: Thay đổi kích thước hình ảnh.
- Xoay, lật: Thay đổi hướng của hình ảnh.
- Biến đổi phối cảnh: Tạo hiệu ứng 3D.

6. Trích xuất đặc trưng:

- SIFT - Scale-Invariant Feature Transform
- SURF: Speeded Up Robust Features
- HOG: Histogram of Oriented Gradients

7. Nhận dạng vật thể:

- Haar cascades: Phát hiện các đối tượng đơn giản như khuôn mặt, mắt.
- Deep learning: Sử dụng mạng thần kinh sâu để nhận dạng các đối tượng phức tạp.

Số hoá ảnh và Nhận biết bước xám

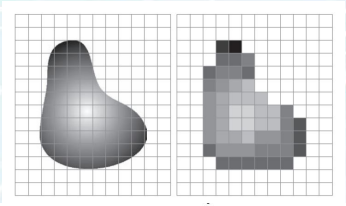
Số hóa ảnh là biến đổi một ảnh (hay một hàm) liên tục trong không gian cũng như theo giá trị thành dạng số rời rạc.

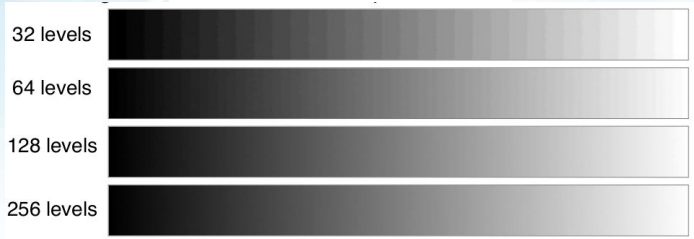
2 bước thực hiện số hoá ảnh:

- Lấy mẫu (sampling): đo giá trị trong các khoảng không gian
- Lượng tử hóa (quantization): ánh xạ cường độ (hoặc giá trị) đo được thành một số hữu hạn các mức rời rạc

Mức độ lượng tử hóa: Số lượng mức xám được yêu cầu trên một bức ảnh số tương ứng với số lượng bits trên mỗi pixel.Ảnh số thường được lượng tử hoá thành 256 mức xám

Ví dụ minh hoạ: Hình ảnh liên tục được chiếu lên một mảng cảm biến và kết quả là lấy mẫu hình ảnh và lượng tử hóa.

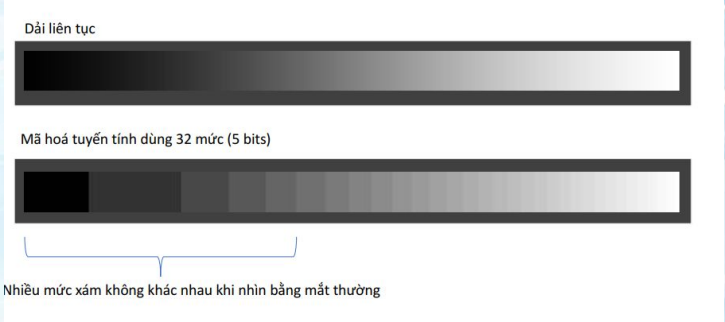




Nhận biết bước xám trong ảnh số là khả năng của các thuật toán và phần mềm phân tích hình ảnh trong việc phân biệt và đo lường các mức độ xám khác nhau trong một bức ảnh. Khả năng này đóng vai trò quan trọng trong nhiều ứng dụng xử lý ảnh, như:

- Cân bằng sáng tối: Điều chỉnh các mức độ sáng tối để tạo ra hình ảnh hài hòa và tự nhiên.
- Phân đoạn ảnh: Tách các đối tượng khỏi nền dựa trên sự khác biệt về mức độ xám.
- Nhận diện vật thể: Tìm kiếm và xác định các vật thể trong ảnh dựa trên các đặc trưng về màu sắc và độ sáng.
- Xử lý ảnh: Các hiệu ứng như tăng cường độ tương phản, làm mờ, hoặc tạo các hiệu ứng nghệ thuật thường dựa trên việc điều chỉnh các mức độ xám.

Nếu mức xám dùng 8 bit nó sẽ là dải liên tục



Toán tử điểm ảnh

Toán tử điểm ảnh là một phép biến đổi ảnh số bằng cách áp dụng một hàm toán học lên từng điểm ảnh riêng lẻ của ảnh hay còn gọi là phương pháp biến đổi pixel để đạt được hiệu ứng hoặc cải thiện chất lượng hình ảnh mong muốn.. Giá trị mới của mỗi điểm ảnh sẽ phụ thuộc vào giá trị gốc của điểm ảnh đó và có thể cả giá trị của các điểm ảnh lân cận.

Mục đích của việc điều chỉnh từng pixel:
- Nâng cao chất lượng hình ảnh: Tăng độ sáng, độ tương phản, cân bằng màu sắc, độ nét.
- Khôi phục hình ảnh: Loại bỏ nhiễu, vết xước, biến dạng.
- Tạo hiệu ứng: Làm mờ, làm sắc nét, phát hiện cạnh, và nhiều hiệu ứng khác.

Cách thức hoạt động
- Mỗi điểm ảnh là một số: Trong máy tính, một hình ảnh được biểu diễn dưới dạng một ma trận các số. Mỗi số tương ứng với giá trị màu của một điểm ảnh.
- Áp dụng hàm toán học: Ta sẽ áp dụng một hàm toán học lên giá trị của mỗi điểm ảnh để tính ra giá trị mới. Hàm này có thể đơn giản như cộng, trừ, nhân, chia hoặc phức tạp hơn như hàm logarit, hàm mũ.
- Tạo ảnh mới: Các giá trị mới của các điểm ảnh sẽ tạo thành một ảnh mới.

Ứng dụng của biến đổi pixel:
- Hình ảnh y tế: Nâng cao hình ảnh để chẩn đoán và phân tích.
- Viễn thám: Xử lý ảnh vệ tinh để phân loại sử dụng đất và theo dõi môi trường.

Các phương pháp biến đổi pixel phổ biến:
1)Biến đổi tuyến tính
- Độ sáng: Cộng thêm một giá trị cố định vào mỗi pixel.
- Độ tương phản: Nhân giá trị mỗi pixel với một hằng số.
- Ảnh âm bản: Trừ giá trị mỗi pixel đi giá trị lớn nhất có thể.

2) Biến đổi phi tuyến
- Hiệu chỉnh Gamma: Điều chỉnh độ sáng và độ tương phản tổng thể của hình ảnh bằng một hàm mũ.
- Cân bằng histogram: Kéo giãn histogram của hình ảnh để cải thiện độ tương phản.
- Biến đổi logarit: Nén dải động của hình ảnh, hữu ích cho hình ảnh có độ tương phản cao.

3) Lọc không gian
- Convolution: Áp dụng một kernel (ma trận) lên mỗi pixel để tạo ra một giá trị pixel mới dựa trên các pixel lân cận.
- Làm mờ: Làm mờ hình ảnh để giảm nhiễu và chi tiết.
- Làm sắc nét: Nâng cao các cạnh và chi tiết.
- Phát hiện cạnh: Xác định ranh giới giữa các vùng khác nhau trong hình ảnh.

Correct using pixel transform

```
def pixel_transform(image, alpha = 1.0, beta = 0):
 '''
 out[pixel] = alpha * image[pixel] + beta
 '''
 output = np.zeros(image.shape, image.dtype)
 h, w, ch = image.shape
 for y in range(h):
   for x in range(w):
     for c in range(ch):
       output[y,x,c] = np.clip(alpha*image[y,x,c] + beta, 0, 255)

 return output
```

```
transformed_high = pixel_transform(high, 0.5, 20)
plt.imshow(transformed_high[:,:,::-1])
```

Các toán tử điểm ảnh tuyến tính

1. Điều chỉnh độ sáng: Tăng hoặc giảm giá trị của mỗi điểm ảnh để làm cho ảnh sáng hơn hoặc tối hơn.
Công thức:

- Tăng độ sáng: pixel_mới = pixel_cũ + giá_trị_tăng
- Giảm độ sáng: pixel_mới = pixel_cũ - giá_trị_giảm

Ví dụ: Giả sử ta có một ảnh xám với các giá trị điểm ảnh từ 0 đến 255. Để tăng độ sáng của ảnh lên 50 đơn vị, ta cộng thêm 50 vào giá trị của mỗi điểm ảnh.

2. Phân ngưỡng: Chuyển đổi ảnh xám thành ảnh nhị phân (đen trắng) bằng cách so sánh giá trị của mỗi điểm ảnh với một ngưỡng nhất định.
Công thức:

- Nếu pixel_cũ >= ngưỡng: pixel_mới = 255 (màu trắng)
- Nếu pixel_cũ < ngưỡng: pixel_mới = 0 (màu đen)

Ví dụ: Chọn ngưỡng là 128. Các điểm ảnh có giá trị lớn hơn hoặc bằng 128 sẽ trở thành màu trắng, còn lại là màu đen.

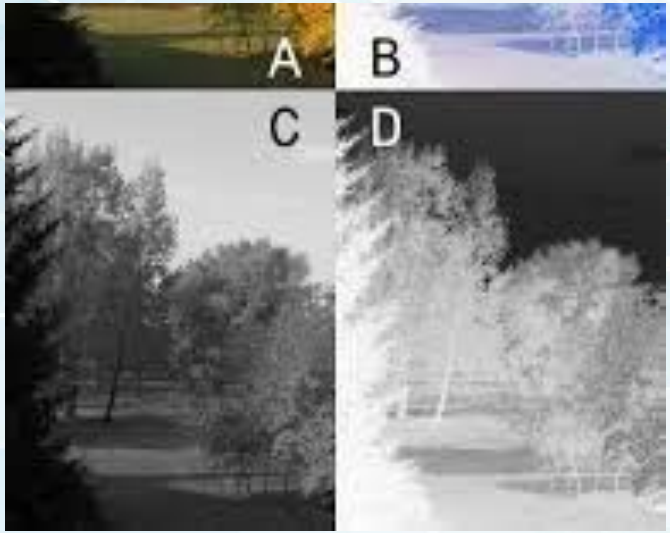
3. Đảo ngược âm bản: Đảo ngược màu sắc của mỗi điểm ảnh.
Công thức:

- pixel_mới = 255 - pixel_cũ

Ví dụ: Nếu một điểm ảnh có giá trị là 50, sau khi đảo ngược sẽ có giá trị là 255 - 50 = 205.

Image Negative (Ảnh âm bản)

Ảnh âm bản là một kỹ thuật trong xử lý ảnh, trong đó các tông màu sáng trong hình ảnh gốc sẽ trở thành tối và ngược lại. Nói cách khác, ảnh âm bản là một phiên bản "đảo ngược" của hình ảnh gốc.




```python
import cv2
import numpy as np

def image_negative(image):
  """Inverts the colors of an image.

  Args:
    image: The input image as a NumPy array.

  Returns:
    The inverted image as a NumPy array.
  """

  # Create a negative image by subtracting each pixel value from 255
  negative_image = 255 - image

  return negative_image

# Load an image
image = cv2.imread('image.jpg')

# Invert the colors of the image
negative_image = image_negative(image)

# Display the original and inverted images
cv2.imshow('Original', image)
cv2.imshow('Negative', negative_image)
cv2.waitKey(0)
```

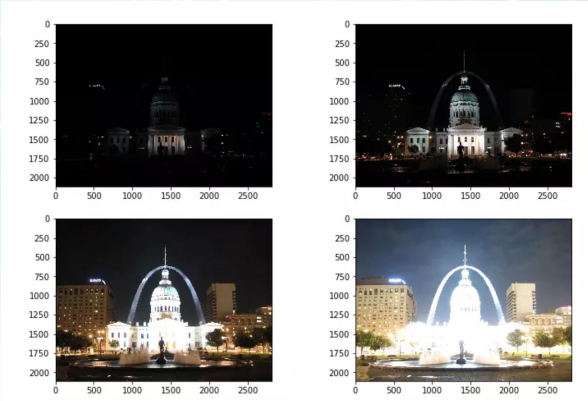
Combine image (Cộng ảnh)

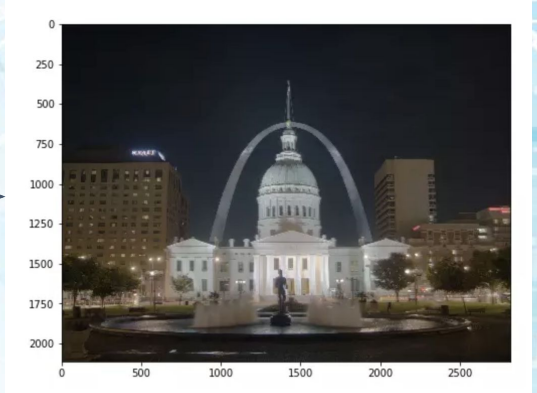
Kỹ thuật kết hợp hình ảnh (hay còn gọi là ghép ảnh, chắp ghép ảnh) là một phương pháp cho phép kết hợp hai hoặc nhiều hình ảnh khác nhau để tạo ra một tác phẩm hoàn toàn mới.

Các phương pháp kết hợp hình ảnh phổ biến:

- Overlay (trồng chồng): Đặt một hình ảnh lên trên hình ảnh khác, tạo ra hiệu ứng trong suốt hoặc mờ dần.
- Masking (tạo mặt nạ): Sử dụng một lớp mặt nạ để che đi một phần của hình ảnh và thay thế bằng một phần của hình ảnh khác.
- Blending (trộn màu): Trộn màu sắc của hai hình ảnh để tạo ra một hiệu ứng chuyển tiếp mượt mà.
- Stitching (ghép nối): Kết hợp nhiều hình ảnh lại với nhau để tạo ra một bức ảnh toàn cảnh.
- Morphing (biến hình): Chuyển đổi dần dần từ một hình ảnh này sang hình ảnh khác.

```python
import cv2

# Load the two images
image1 = cv2.imread("image1.jpg")
image2 = cv2.imread("image2.jpg")

# Resize the images to the same size
image1 = cv2.resize(image1, (500, 500))
image2 = cv2.resize(image2, (500, 500))

# Combine the images using the overlay method
combined_image = cv2.addWeighted(image1, 0.5, image2, 0.5, 0)

# Display the combined image
cv2.imshow("Combined image", combined_image)
cv2.waitKey(0)
```





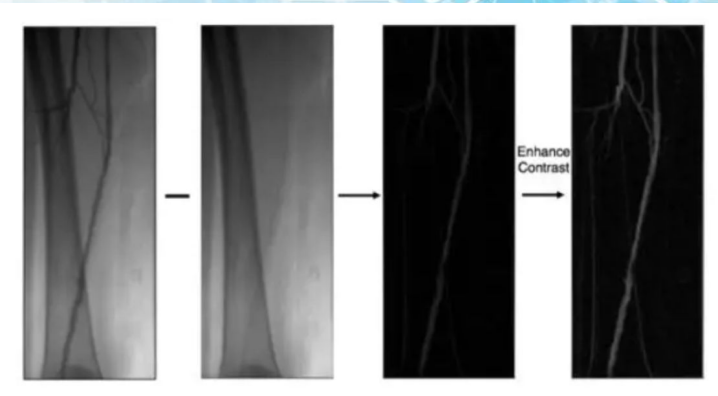
Phép trừ ảnh

Phép trừ ảnh là một kỹ thuật xử lý ảnh cơ bản, thường được sử dụng để tìm ra những điểm khác biệt giữa hai hình ảnh. Khi thực hiện phép trừ, các pixel tương ứng trong hai hình ảnh sẽ được trừ cho nhau. Kết quả thu được sẽ là một hình ảnh mới, trong đó các điểm sáng thể hiện những vùng có sự khác biệt lớn giữa hai hình ảnh gốc.

Mục đích sử dụng phép trừ ảnh:
- Phát hiện thay đổi: Phép trừ ảnh thường được dùng để phát hiện những thay đổi nhỏ giữa hai hình ảnh liên tiếp, ví dụ như trong các hệ thống giám sát, theo dõi chuyển động.
- Phân tích hình ảnh: Bằng cách trừ đi một hình ảnh nền, chúng ta có thể tập trung vào các đối tượng chuyển động hoặc các chi tiết quan trọng khác.
- Xử lý ảnh: Phép trừ ảnh có thể được sử dụng như một bước tiền xử lý trong nhiều thuật toán xử lý ảnh khác.

Cách thức thực hiện phép trừ ảnh:
- Kiểm tra kích thước: Đảm bảo hai hình ảnh có cùng kích thước. Nếu không, cần thực hiện resize trước khi trừ.
- Trừ từng pixel: Mỗi pixel trong hình ảnh kết quả sẽ bằng giá trị pixel tương ứng trong hình ảnh A trừ đi giá trị pixel tương ứng trong hình ảnh B.
- Hiển thị kết quả: Hình ảnh kết quả sẽ cho thấy những vùng sáng ở nơi có sự khác biệt lớn giữa hai hình ảnh gốc.

```python
import cv2
import numpy as np

# Đọc hai hình ảnh
img1 = cv2.imread('image1.jpg')
img2 = cv2.imread('image2.jpg')

# Chuyển đổi hình ảnh thành grayscale
gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

# Thực hiện phép trừ
diff = cv2.absdiff(gray1, gray2)

# Hiển thị kết quả
cv2.imshow('Difference', diff)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Trừ nền trong video (Video Background Subtraction)

Trừ nền trong video là một kỹ thuật mở rộng từ phép trừ ảnh, được áp dụng để phát hiện các đối tượng chuyển động trong một chuỗi các hình ảnh liên tiếp (video). Kỹ thuật này rất hữu ích trong các ứng dụng giám sát, theo dõi hành vi, và nhiều lĩnh vực khác.

Mục đích:
- Phát hiện chuyển động: Nhận diện các đối tượng đang di chuyển trong một cảnh tĩnh.
- Theo dõi đối tượng: Đánh dấu và theo dõi các đối tượng đã được phát hiện.
- Phân tích hành vi: Dựa trên các chuyển động để phân tích hành vi của các đối tượng.

Các phương pháp trừ nền phổ biến:
- Trừ nền đơn giản: Trừ từng frame của video với một hình ảnh nền tĩnh được chọn trước. Phương pháp này đơn giản nhưng dễ bị ảnh hưởng bởi các thay đổi về ánh sáng.
- Trừ nền thích ứng: Tạo một mô hình nền động, liên tục cập nhật để thích ứng với các thay đổi về ánh sáng và các yếu tố môi trường khác.

Các thuật toán phổ biến:
- ViBe: Dựa trên các mẫu màu để mô hình hóa nền.
- KNN: Sử dụng k-nearest neighbors để phân loại các pixel.
- MOG2: Mô hình hỗn hợp Gaussian để mô hình hóa sự phân bố màu sắc của nền.
- Optical Flow: Tính toán sự dịch chuyển của các pixel giữa hai frame liên tiếp để phát hiện chuyển động.

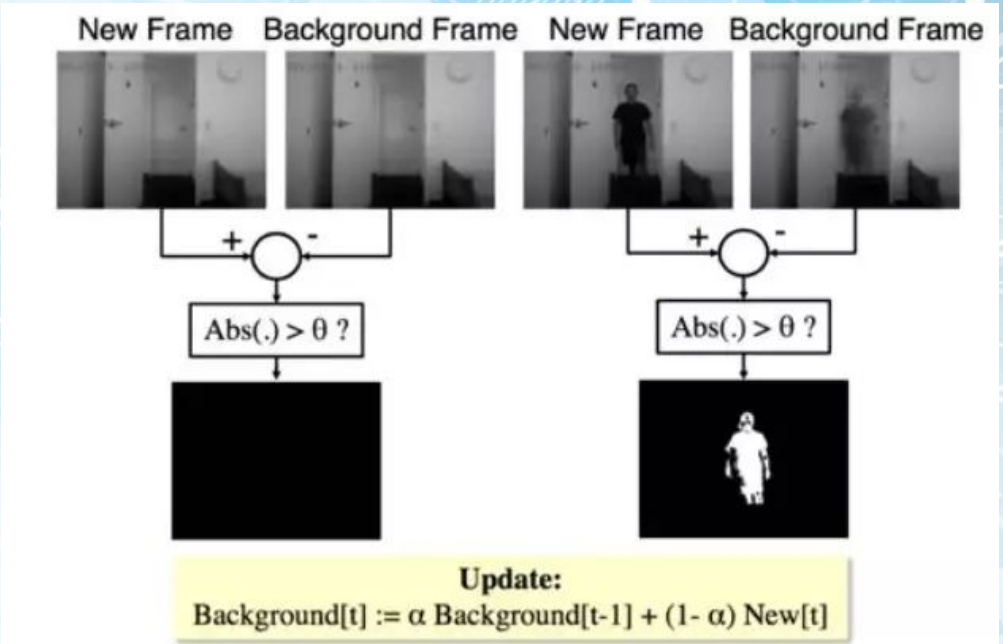
Thách thức trong việc trừ nền video
- Ánh sáng thay đổi: Ánh sáng thay đổi có thể ảnh hưởng đến hiệu quả của việc trừ nền.
- Bóng: Bóng của các đối tượng chuyển động có thể gây nhiễu.
- Đối tượng mờ: Các đối tượng chuyển động nhanh hoặc mờ có thể khó phát hiện.
- Nền phức tạp: Nền có nhiều chi tiết hoặc thay đổi liên tục sẽ làm khó khăn cho việc phân biệt đối tượng và nền.

Ứng dụng của trừ nền video
- Hệ thống giám sát: Phát hiện xâm nhập, theo dõi phương tiện giao thông.
- Xe tự lái: Phát hiện người đi bộ, xe cộ.
- Hệ thống tương tác: Nhận diện cử chỉ, theo dõi mắt.
- Phân tích video: Tính toán lưu lượng giao thông, phân tích hành vi người dùng.

Trừ nền trong video (Video Background Subtraction)

```python
import cv2

# Tạo một đối tượng để trừ nền
fgbg = cv2.createBackgroundSubtractorMOG2()

# Đọc video
cap = cv2.VideoCapture('video.mp4')

while(True):
    ret, frame = cap.read()

    # Áp dụng thuật toán trừ nền
    fgmask = fgbg.apply(frame)

    # Hiển thị kết quả
    cv2.imshow('Frame', frame)
    cv2.imshow('FG Mask', fgmask)

    k = cv2.waitKey(30) & 0xff
    if k == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

Trong ví dụ bên, thuật toán MOG2 được sử dụng để trừ nền. Mỗi frame của video sẽ được so sánh với mô hình nền và tạo ra một mask (mặt nạ) chỉ ra các vùng có đối tượng chuyển động.



Biến đổi Gamma

Gamma, trong ngữ cảnh xử lý hình ảnh, là một thuật ngữ dùng để mô tả mối quan hệ phi tuyến giữa giá trị đầu vào (tín hiệu) và giá trị đầu ra (độ sáng) của một thiết bị hiển thị như màn hình máy tính. Nói một cách đơn giản, gamma quyết định cách các màu sắc sáng tối được hiển thị trên màn hình.

Biến đổi Gamma là một phép toán phi tuyến được sử dụng rộng rãi trong xử lý ảnh kỹ thuật số. Nó có tác dụng điều chỉnh độ tương phản của một hình ảnh bằng cách thay đổi mối quan hệ giữa giá trị đầu vào (pixel gốc) và giá trị đầu ra (pixel sau khi biến đổi).

Tại sao cần biến đổi Gamma?

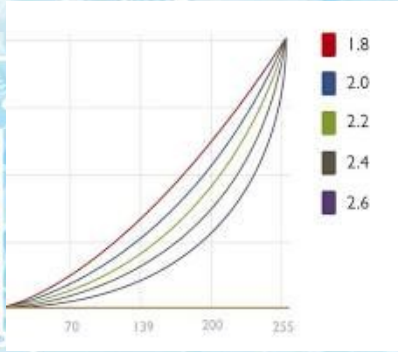
- Hiệu chỉnh độ sáng: Mỗi thiết bị hiển thị (màn hình, máy in) có đặc tính gamma khác nhau. Biến đổi gamma giúp điều chỉnh để hình ảnh hiển thị đúng với ý định của người tạo.
- Tăng cường độ tương phản: Đối với các hình ảnh có độ tương phản thấp, biến đổi gamma có thể giúp làm nổi bật các chi tiết sáng tối.
- Điều chỉnh màu sắc: Mặc dù chủ yếu ảnh hưởng đến độ sáng, biến đổi gamma cũng có thể gián tiếp ảnh hưởng đến màu sắc của hình ảnh.

Ứng dụng của biến đổi Gamma

- Hiệu chỉnh màu sắc: Đảm bảo hình ảnh hiển thị đúng trên các thiết bị khác nhau.
- Xử lý ảnh: Tăng cường độ tương phản, điều chỉnh độ sáng, tạo các hiệu ứng đặc biệt.
- Đồ họa máy tính: Tạo ra các hiệu ứng ánh sáng, bóng đổ, và các hiệu ứng hình ảnh khác.
- Xử lý video: Cân bằng màu sắc và độ sáng trong video.

Cách Gamma hoạt động

- *Đường cong gamma*: Mối quan hệ giữa giá trị đầu vào và giá trị đầu ra thường được biểu diễn bằng một đường cong gamma. Đường cong này có thể là tuyến tính (gamma = 1) hoặc phi tuyến (gamma khác 1).
- *Gamma thấp*: Đường cong gamma sẽ dẹt hơn, làm giảm độ tương phản.
- *Gamma cao*: Đường cong gamma sẽ cong hơn, tăng độ tương phản.



Sử dụng Gamma để điều chỉnh độ tương phản

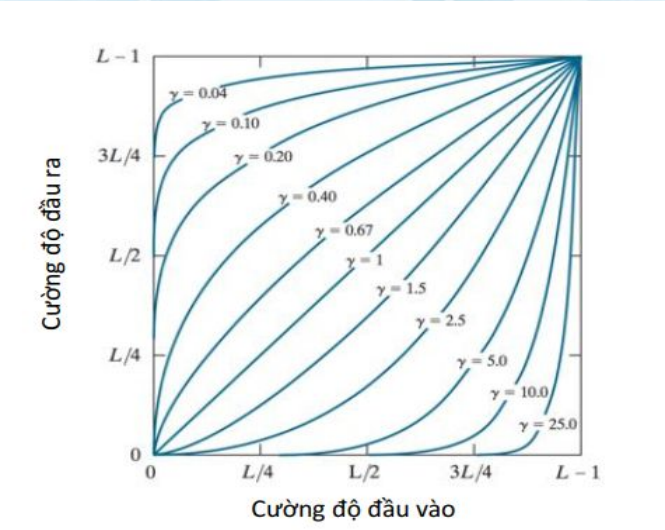
Công thức biến đổi Gamma có dạng:

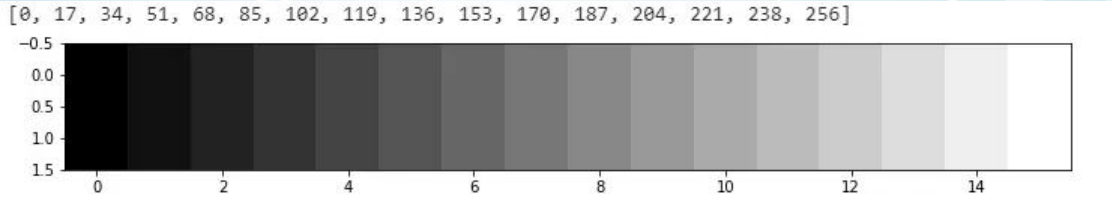
V_out = V_in^gamma

trong đó:

- V_in: Giá trị pixel gốc (trong khoảng từ 0 đến 1)
- V_out: Giá trị pixel sau khi biến đổi
- gamma: Tham số gamma, quyết định mức độ biến đổi.
  - Gamma < 1: Tăng độ tương phản ở vùng sáng, làm tối các vùng tối.
  - Gamma > 1: Tăng độ tương phản ở vùng tối, làm sáng các vùng sáng.
  - Gamma = 1: Không có thay đổi nào.

.





Hình ảnh có độ tương phản cao là những bức ảnh có sự khác biệt rõ rệt giữa vùng sáng nhất và vùng tối nhất.

Độ tương phản cao thường làm tăng độ bão hòa của màu sắc, khiến chúng trở nên sống động và nổi bật hơn.

Các chi tiết trong vùng sáng và tối đều được làm nổi bật, giúp người xem dễ dàng nhận biết và phân biệt chúng

Hình ảnh có độ tương phản thấp, khiến cho các chi tiết khó nhìn rõ.

Màu sắc nhạt nhòa, không có sự tương phản rõ ràng giữa các vùng sáng và tối.

Hình ảnh này có thể được cải thiện bằng cách tăng độ tương phản.

Sử dụng Gamma để điều chỉnh độ tương phản

Ví dụ xây dựng hàm adjust_image_gamma với thư viện cv2 và PIL

```python
import cv2

def adjust_image_gamma(image, gamma):
  """Adjusts the gamma of an image.

  Args:
    image: The input image as a NumPy array.
    gamma: The gamma value to apply.

  Returns:
    The gamma-adjusted image.
  """

  # Convert the image to floating-point data type for calculations
  image = image.astype(float) / 255.0

  # Apply the gamma correction formula
  adjusted_image = (image ** gamma) * 255

  # Convert the image back to integer data type for display
  adjusted_image = adjusted_image.astype(int)

  return adjusted_image
```

```python
from PIL import Image

def adjust_image_gamma_pil(image, gamma):
  """Adjusts the gamma of an image using PIL.

  Args:
    image: The input PIL Image object.
    gamma: The gamma value to apply.

  Returns:
    The gamma-adjusted PIL Image object.
  """

  # Convert the image to floating-point data type for calculations
  image = image.convert('F')

  # Apply the gamma correction formula
  adjusted_image = image ** gamma

  # Convert the image back to integer data type for display
  adjusted_image = adjusted_image.convert('RGB')

  return adjusted_image
```

Histogram

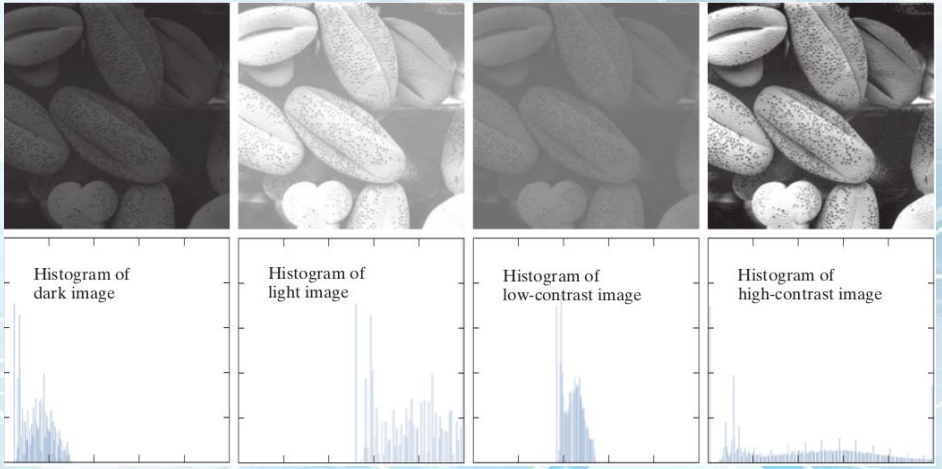
Histogram là một biểu đồ thống kê dùng để biểu diễn phân phối tần suất của dữ liệu liên tục. Trong xử lý ảnh, histogram cho ta biết mỗi mức xám (hoặc mỗi màu trong không gian màu khác) xuất hiện bao nhiêu lần trong một hình ảnh. Nói cách khác, histogram giúp chúng ta hiểu rõ về sự phân bố các giá trị pixel trong một hình ảnh.

Mục đích sử dụng Histogram:

- Phân tích hình ảnh: Histogram giúp chúng ta đánh giá độ tương phản, độ sáng tối, sự phân bố màu sắc của một hình ảnh.
- Cải thiện hình ảnh: Nhiều kỹ thuật xử lý ảnh như cân bằng histogram, tăng cường độ tương phản đều dựa trên thông tin từ histogram.
- So sánh hình ảnh: So sánh histogram của hai hình ảnh để đánh giá mức độ tương đồng.

Các loại Histogram:

- Histogram mức xám: Biểu diễn phân bố các mức xám trong một hình ảnh grayscale.
- Histogram màu: Biểu diễn phân bố các màu trong một hình ảnh màu (ví dụ: RGB, HSV).
- Histogram cục bộ: Tính histogram cho các vùng nhỏ trong hình ảnh.

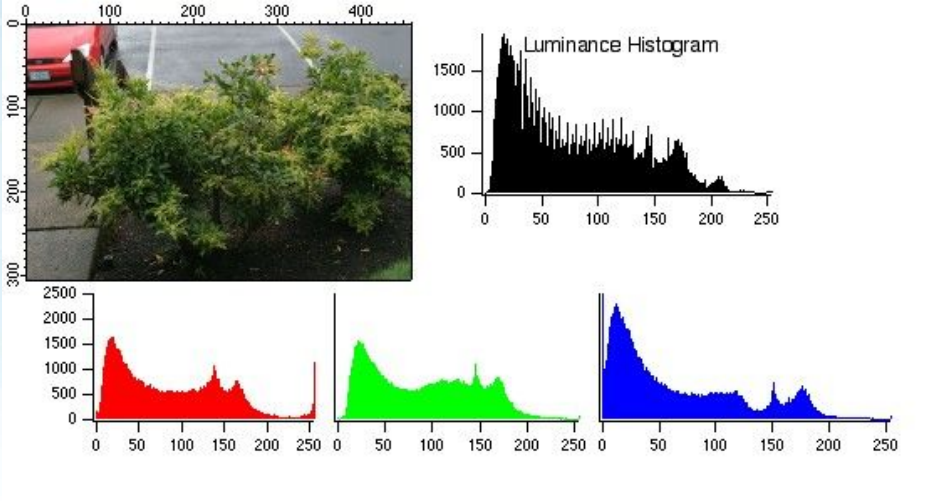


- Với ảnh dark thì histogram có các cột tập trung vào bên trái tương ứng với màu tối
- Với ảnh light thì histogram có tập trung vào bên phải chứa các pixel trắng
- Với ảnh độ tương phản thấp (low-contrast) thì histogram có các cột tập trung xít nhau và ở giữa
- Với ảnh độ tương phản cao (high-contrast) thì histogram san đều với các giá trị

Histogram

Cách tính Histogram:

- Xác định số lượng bins: Một ***bin*** là một khoảng giá trị. Số lượng bins ảnh hưởng đến độ chi tiết của histogram.
- Khởi tạo mảng đếm: Tạo một mảng có kích thước bằng số lượng bins, tất cả các phần tử đều bằng 0.
- Đếm số lượng pixel: Duyệt qua từng pixel trong ảnh.
- Xác định giá trị pixel thuộc vào bin nào.
- Tăng giá trị của ***bin*** tương ứng lên 1.
- Vẽ histogram: Biểu diễn kết quả đếm dưới dạng một biểu đồ cột, trong đó trục hoành là giá trị pixel và trục tung là tần suất xuất hiện.



Ví dụ bằng Python và OpenCV

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Đọc ảnh
img = cv2.imread('image.jpg', 0)  # Đọc ảnh xám

# Tính histogram
hist = cv2.calcHist([img], [0], None, [256], [0, 256])

# Vẽ histogram
plt.plot(hist)
plt.xlabel('Intensity')
plt.ylabel('Number of pixels')
plt.show()
```

cv2.calcHist là hàm dùng để tính histogram.
[img]: Danh sách các hình ảnh (ở đây chỉ có 1).
[0]: Chỉ định tính histogram cho kênh màu thứ 0 (grayscale).
None: Không sử dụng mask.
[256]: Số lượng bins.
[0, 256]: Khoảng giá trị của bins.

Histogram equalization

Cân bằng histogram là một kỹ thuật xử lý ảnh nhằm tăng cường độ tương phản của một hình ảnh bằng cách phân phối lại các mức xám trong histogram. Điều này giúp cho hình ảnh trở nên sáng hơn, rõ nét hơn và chi tiết hơn, đặc biệt là trong các trường hợp hình ảnh có độ tương phản thấp hoặc các vùng sáng tối phân bố không đều.

Mục đích:

- Tăng cường độ tương phản: Làm cho các chi tiết trong hình ảnh trở nên rõ ràng hơn, đặc biệt là trong các vùng bóng tối hoặc vùng sáng quá sáng.
- Cải thiện chất lượng hình ảnh: Làm cho hình ảnh trông hấp dẫn hơn về mặt thị giác.
- Chuẩn bị cho các bước xử lý tiếp theo: Histogram cân bằng thường là một bước tiền xử lý cần thiết cho các thuật toán xử lý ảnh khác như nhận dạng đối tượng, phân đoạn ảnh.

Ưu điểm:

- Đơn giản, dễ thực hiện.
- Hiệu quả trong nhiều trường hợp.
- Không yêu cầu thông tin về nội dung hình ảnh.

Nhược điểm:

- Có thể làm tăng nhiễu trong hình ảnh.
- Không phù hợp với tất cả các loại hình ảnh, đặc biệt là các hình ảnh có nhiều chi tiết nhỏ.

Nguyên lý hoạt động

- Tính toán histogram: Tính histogram của hình ảnh gốc để biết phân bố các mức xám.
- Tạo hàm ánh xạ: Tạo một hàm ánh xạ mới, trong đó các mức xám xuất hiện ít sẽ được ánh xạ sang các mức xám sáng hơn, và các mức xám xuất hiện nhiều sẽ được ánh xạ sang các mức xám tối hơn.
- Áp dụng hàm ánh xạ: Áp dụng hàm ánh xạ này cho từng pixel trong hình ảnh để tạo ra hình ảnh mới.

Công thức:

$$s = T(r)$$

trong đó:

- s: Mức xám mới của pixel
- r: Mức xám cũ của pixel
- T: Hàm ánh xạ

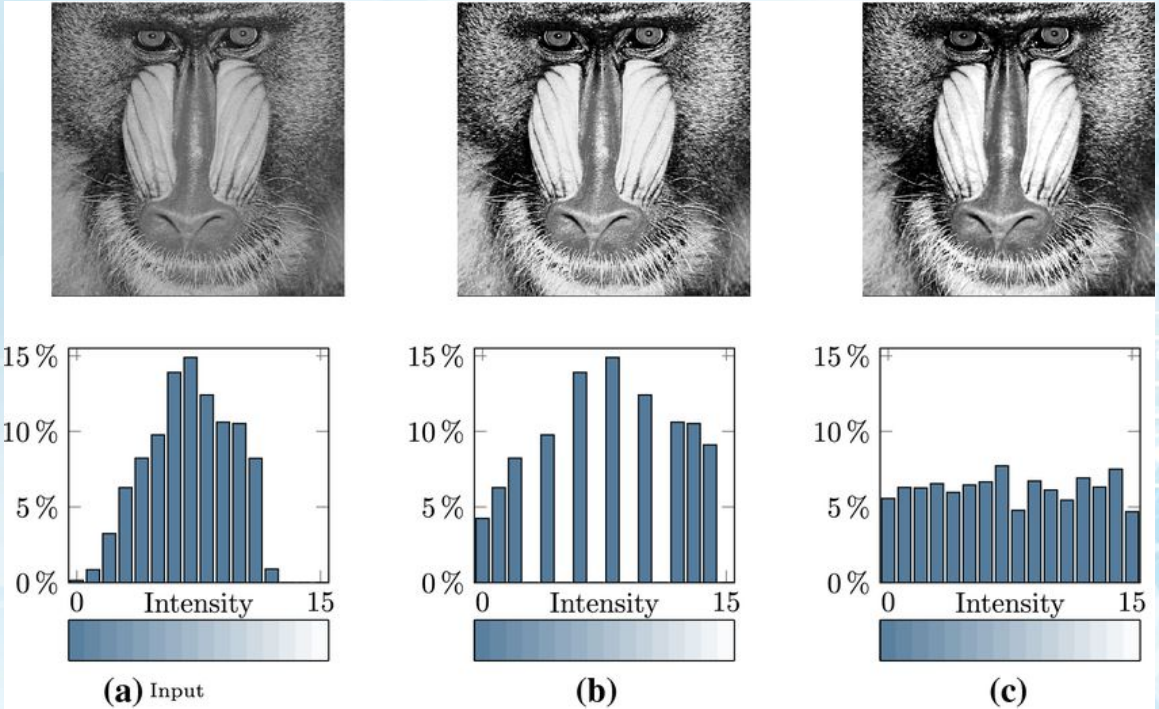
Hàm ánh xạ thường được chọn là:

$$T(r) = (L-1) * \int_{[0,r]} p(x)dx$$

trong đó:

- L: Số lượng mức xám (thường là 256)
- p(x): Xác suất xuất hiện của mức xám x

Histogram equalization




```python
import cv2
import numpy as np

# Đọc ảnh
img = cv2.imread('image.jpg', 0)

# Cân bằng histogram
equ = cv2.equalizeHist(img)

# Hiển thị kết quả
cv2.imshow('Original', img)
cv2.imshow('Equalized', equ)
cv2.waitKey(0)
```

Toán tử láng giềng

Toán tử láng giềng là một khái niệm nền tảng trong xử lý ảnh. Nó giúp chúng ta khai thác thông tin từ các pixel lân cận để thực hiện các phép biến đổi và phân tích hình ảnh.

Láng giềng của một pixel là tập hợp các pixel xung quanh pixel đó.

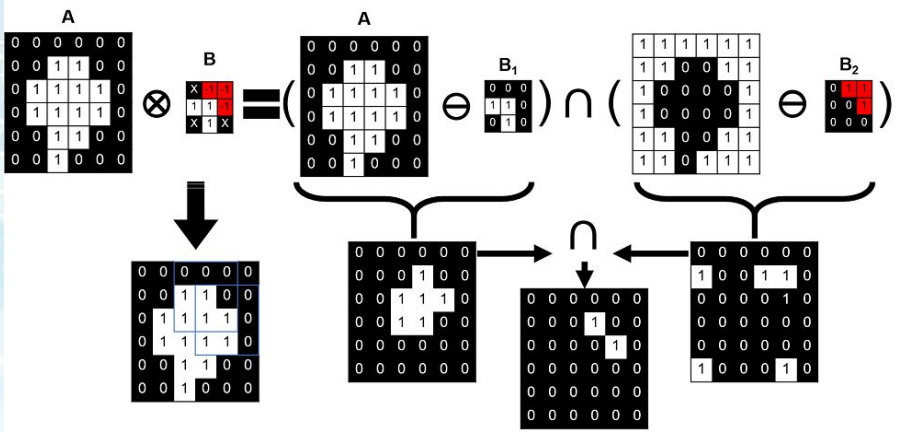
Toán tử láng giềng là một phép toán được áp dụng lên một pixel và các láng giềng của nó để tính toán một giá trị mới.

Các loại láng giềng phổ biến

- Láng giềng 4-kết nối: Gồm các pixel ở trên, dưới, trái, phải của pixel đang xét.
- Láng giềng 8-kết nối: Bao gồm cả các pixel ở các đường chéo.
- Láng giềng k-kết nối: Là một khái niệm tổng quát, bao gồm các pixel nằm trong một vùng hình tròn có bán kính k xung quanh pixel đang xét.

Ứng dụng của toán tử láng giềng trong xử lý ảnh

1. *Phát hiện cạnh*. Các cạnh trong ảnh thường có sự thay đổi đột ngột về cường độ sáng. Bằng cách so sánh giá trị của một pixel với các láng giềng, ta có thể phát hiện ra các cạnh. Các toán tử: Sobel, Prewitt, Laplacian.
2. *Làm mờ ảnh.* Có thể làm giảm nhiễu và làm mờ các chi tiết nhỏ trong ảnh bằng cách tính trung bình giá trị của một pixel và các láng giềng của nó. Các toán tử: Trung bình, Gaussian blur.
3. *Phân đoạn ảnh* là nhóm các pixel có đặc trưng tương tự nhau (ví dụ: màu sắc, cường độ) thành các vùng. Các toán tử: Region growing, watershed.
4. *Tăng cường ảnh* - tăng cường các đặc trưng mong muốn của ảnh, ví dụ như độ tương phản, độ sắc nét. Các toán tử: Histogram equalization, unsharp masking.



Các phép lọc trong không gian điểm ảnh (Spatial Filtering)

Spatial Filtering (Lọc không gian) là một kỹ thuật cơ bản và quan trọng trong xử lý ảnh số. Nó hoạt động bằng cách thay thế giá trị của mỗi pixel trong ảnh bằng một hàm tính toán dựa trên giá trị của pixel đó và các pixel lân cận. Các hàm tính toán này thường được biểu diễn dưới dạng một ma trận, gọi là kernel hoặc mask.

Mục đích:
- Loại bỏ nhiễu: Loại bỏ các điểm ảnh nhiễu, làm cho hình ảnh mịn màng hơn.
- Làm mờ: Làm mờ các cạnh và chi tiết nhỏ trong ảnh.
- Tăng cường cạnh: Phát hiện và làm nổi bật các cạnh trong ảnh.
- Nhận dạng mẫu: Tìm kiếm các mẫu nhất định trong ảnh.

Ví dụ về kernel lọc:

Loại lọc	Kernel	Mô tả
Trung bình 3x3	[[1/9, 1/9, 1/9], [1/9, 1/9, 1/9], [1/9, 1/9, 1/9]]	Trung bình cộng các pixel trong vùng 3x3
Gaussian 3x3	[[1/16, 2/16, 1/16], [2/16, 4/16, 2/16], [1/16, 2/16, 1/16]]	Lọc Gaussian với sigma nhỏ
Laplacian 3x3	[[-1, -1, -1], [-1, 8, -1], [-1, -1, -1]]	Phát hiện cạnh

Ứng dụng của lọc không gian:
- Xử lý ảnh: Loại bỏ nhiễu, tăng cường cạnh, làm mờ, phát hiện đối tượng.
- Xử lý video: Làm mịn video, ổn định video.
- Nhận dạng hình ảnh: Trích xuất các đặc trưng hình ảnh.
- Xử lý ảnh y tế: Tăng cường các chi tiết trong hình ảnh y tế.

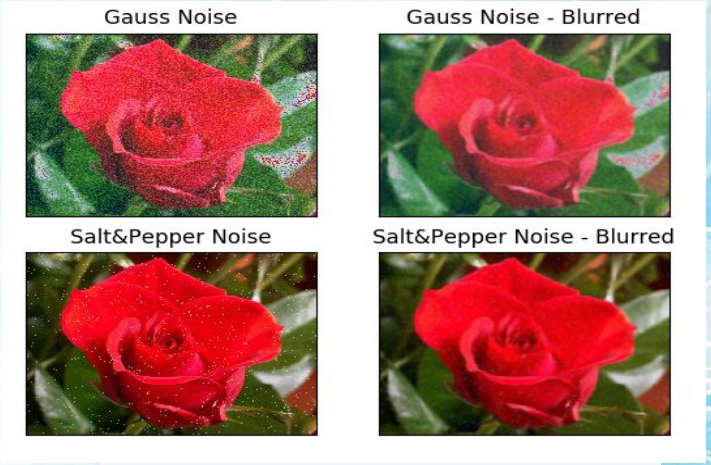
Các loại lọc không gian phổ biến

1. Lọc tuyến tính

- Lọc trung bình: Mỗi pixel mới được tính bằng trung bình cộng của giá trị pixel đó và các pixel lân cận. Giúp làm mờ ảnh, loại bỏ nhiễu nhưng cũng làm mất chi tiết.
- Lọc Gaussian: Sử dụng kernel Gaussian để làm mờ ảnh, các pixel gần tâm kernel có trọng số lớn hơn. Giữ lại các cạnh tốt hơn so với lọc trung bình.
- Lọc Laplacian: Tính toán đạo hàm bậc hai của ảnh, giúp phát hiện các cạnh và chi tiết. Kernel Laplacian thường có các giá trị dương ở tâm và các giá trị âm xung quanh.

2. Lọc phi tuyến tính

- Lọc trung vị: Thay thế mỗi pixel bằng trung vị của các giá trị pixel trong một vùng lân cận. Hiệu quả trong việc loại bỏ nhiễu muối tiêu.
- Lọc Min/Max: Thay thế mỗi pixel bằng giá trị nhỏ nhất hoặc lớn nhất trong vùng lân cận. Sử dụng để phát hiện các đối tượng sáng hoặc tối.
- Lọc Bilateral: Kết hợp cả thông tin không gian và cường độ để làm mờ ảnh. Giữ lại các cạnh trong khi làm mờ các vùng đồng nhất.




```python
import cv2
import numpy as np

# Đọc ảnh
img = cv2.imread('image.jpg', 0)

# Tạo kernel lọc
kernel = np.ones((5,5), np.float32)/25

# Áp dụng lọc
dst = cv2.filter2D(img,-1,kernel)

# Hiển thị kết quả
cv2.imshow('Original', img)
cv2.imshow('Filtered', dst)
cv2.waitKey(0)
```

Lọc tuyến tính trong xử lý ảnh (linear processing filtering)

Lọc tuyến tính là một kỹ thuật cơ bản và phổ biến trong xử lý ảnh số. Nó hoạt động bằng cách thay thế giá trị của mỗi pixel trong ảnh bằng một tổ hợp tuyến tính của giá trị pixel đó và các pixel lân cận. Tổ hợp tuyến tính này được xác định bởi một ma trận số gọi là kernel hoặc mask.

Mục đích:
- Loại bỏ nhiễu: Giảm thiểu nhiễu Gaussian, muối tiêu.
- Làm mờ: Làm mịn ảnh, giảm chi tiết.
- Tăng cường cạnh: Phát hiện và làm nổi bật các cạnh trong ảnh.
- Nhận dạng mẫu: Tìm kiếm các mẫu nhất định trong ảnh.

Nguyên lý hoạt động:
- Kernel: Kernel là một ma trận nhỏ chứa các hệ số trọng số. Mỗi phần tử trong kernel tương ứng với một pixel trong vùng lân cận của pixel đang xét.
- Cuộn kernel: Kernel được dịch chuyển qua từng pixel của ảnh.
- Tính toán giá trị pixel mới: Tại mỗi vị trí, giá trị pixel mới được tính bằng cách nhân từng phần tử của kernel với giá trị pixel tương ứng trong vùng lân cận, sau đó cộng tất cả các kết quả lại.

Các loại lọc tuyến tính phổ biến

- **Lọc trung bình:**
  - Tất cả các phần tử trong kernel có giá trị bằng nhau.
  - Giúp làm mờ ảnh, loại bỏ nhiễu.
- **Lọc Gaussian:**
  - Kernel có dạng phân phối Gaussian.
  - Giữ lại các cạnh tốt hơn so với lọc trung bình.
- **Lọc Laplacian:**
  - Kernel có các giá trị dương ở tâm và các giá trị âm xung quanh.
  - Phát hiện các cạnh và chi tiết.

Ưu điểm và nhược điểm

- **Ưu điểm:**
  - Đơn giản, dễ thực hiện.
  - Hiệu quả cao.
  - Có nhiều loại kernel khác nhau để lựa chọn.
- **Nhược điểm:**
  - Có thể làm mất một số chi tiết của ảnh.
  - Không hiệu quả với các loại nhiễu phức tạp.

Lọc trung bình

Lọc trung bình là một kỹ thuật cơ bản trong xử lý ảnh, được sử dụng để làm mịn hình ảnh và giảm nhiễu. Nguyên lý hoạt động của nó khá đơn giản:

- **Mặt nạ lọc:** Một ma trận số học nhỏ, thường là hình vuông (ví dụ: 3x3, 5x5), được đặt lên từng điểm ảnh trong hình ảnh.
- **Tính trung bình:** Giá trị tại điểm ảnh trung tâm của mặt nạ sẽ được thay thế bằng giá trị trung bình của tất cả các điểm ảnh nằm trong mặt nạ.
- **Trượt mặt nạ:** Mặt nạ được di chuyển lần lượt qua từng điểm ảnh của hình ảnh để thực hiện quá trình tính trung bình.

Ví dụ:

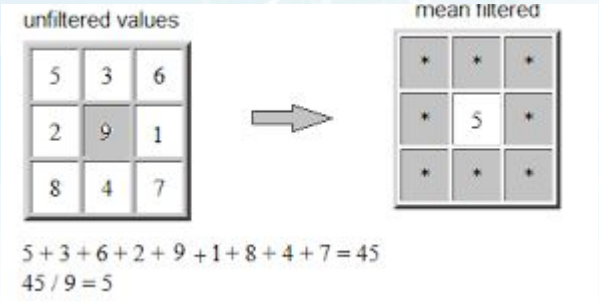
Giả sử ta có một mặt nạ lọc 3x3 và một phần hình ảnh như sau:

$$\begin{matrix} 5 & 3 & 2 \\ 7 & 9 & 4 \\ 6 & 8 & 1 \end{matrix}$$

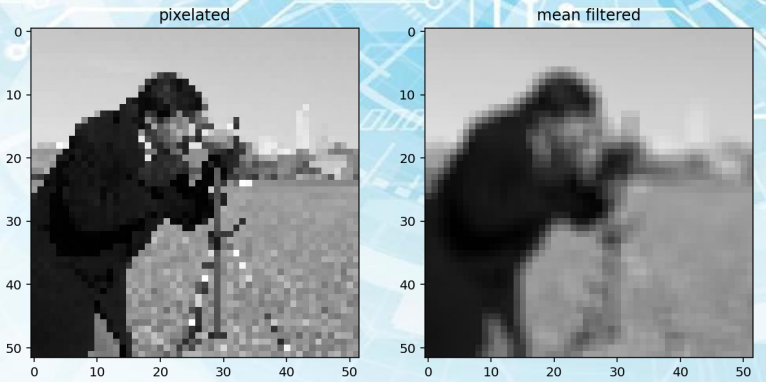
Giá trị trung bình của các điểm ảnh trong mặt nạ sẽ là:

$$(5+3+2+7+9+4+6+8+1)/9 = 5$$

Vậy điểm ảnh trung tâm (giá trị 9) sẽ được thay thế bằng giá trị 5.




```python
import cv2
import numpy as np

def median_filter(image, kernel_size=3):
    """
    Applies median filtering to an image using OpenCV.

    Args:
        image: The input image as a NumPy array.
        kernel_size: The size of the median filtering kernel (mus

    Returns:
        The filtered image as a NumPy array.
    """

    if kernel_size % 2 == 0:
        raise ValueError("Kernel size must be an odd integer.")

    return cv2.medianBlur(image, kernel_size)

# Example usage
image = cv2.imread('image.jpg')

filtered_image = median_filter(image, kernel_size=3)

cv2.imshow('Original', image)
cv2.imshow('Filtered', filtered_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Lọc Gaussian

Lọc Gaussian là một kỹ thuật xử lý ảnh thường được sử dụng để làm mịn hình ảnh và giảm nhiễu một cách hiệu quả hơn so với lọc trung bình đơn giản. Khác với lọc trung bình, lọc Gaussian sử dụng một ma trận trọng số (kernel) có các giá trị được xác định bởi phân phối Gaussian.

Nguyên lý hoạt động

- Kernel Gaussian: Kernel Gaussian là một ma trận vuông, các giá trị trong ma trận này giảm dần theo khoảng cách tới tâm. Các giá trị ở gần tâm có trọng số lớn hơn, tức là ảnh hưởng nhiều hơn đến giá trị điểm ảnh kết quả.
- Quá trình lọc: Tương tự như lọc trung bình, kernel Gaussian được trượt qua từng điểm ảnh của hình ảnh. Tại mỗi vị trí, giá trị của điểm ảnh trung tâm được tính bằng tổng trọng số của các điểm ảnh lân cận nhân với giá trị tương ứng trong kernel.

```python
import cv2
import numpy as np

def gaussian_filter(img, kernel_size=5):
    return cv2.GaussianBlur(img, (kernel_size, kernel_size), 0)

# Đọc ảnh
img = cv2.imread('image.jpg')

# Áp dụng lọc Gaussian
filtered_img = gaussian_filter(img, 5)

# Hiển thị ảnh
cv2.imshow('Original', img)
cv2.imshow('Filtered', filtered_img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

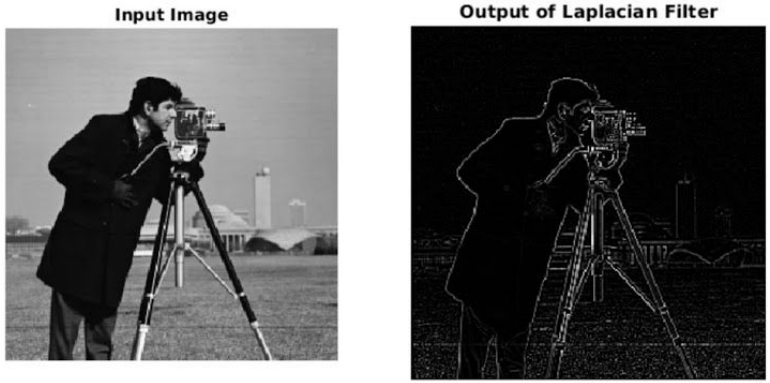
Lọc Laplacian

Lọc Laplacian là một kỹ thuật xử lý ảnh được sử dụng để phát hiện các cạnh trong hình ảnh. Nó dựa trên nguyên tắc rằng các cạnh trong hình ảnh thường có sự thay đổi đột ngột về cường độ xám.

Nguyên lý hoạt động

- **Kernel Laplacian:** Kernel Laplacian là một ma trận vuông, thường có kích thước 3x3. Các giá trị trong ma trận này được thiết kế để tính đạo hàm bậc hai của cường độ xám tại mỗi điểm ảnh.
- **Quá trình lọc:** Kernel Laplacian được trượt qua từng điểm ảnh của hình ảnh. Tại mỗi vị trí, giá trị của điểm ảnh trung tâm được tính bằng tổng trọng số của các điểm ảnh lân cận nhân với giá trị tương ứng trong kernel.
- **Phát hiện cạnh:** Các điểm ảnh có giá trị lớn trong kết quả lọc thường là các điểm nằm trên cạnh của hình ảnh.

```python
import cv2
import numpy as np

def laplacian_filter(img):
    return cv2.Laplacian(img, cv2.CV_64F)

# Đọc ảnh
img = cv2.imread('image.jpg', 0)

# Áp dụng lọc Laplacian
filtered_img = laplacian_filter(img)

# Hiển thị ảnh
cv2.imshow('Original', img)
cv2.imshow('Filtered', filtered_img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Gradient and edge

Gradient. Trong xử lý ảnh, gradient được hiểu đơn giản là độ dốc của cường độ pixel tại một điểm ảnh nhất định. Nó cho biết mức độ thay đổi của cường độ sáng khi di chuyển từ điểm ảnh đó đến các điểm ảnh lân cận. Gradient thường được biểu diễn dưới dạng một vector, chỉ ra hướng và độ lớn của sự thay đổi này.

Mục đích sử dụng Gradient:

- Phát hiện cạnh: Các cạnh trong một hình ảnh thường là nơi có sự thay đổi đột ngột về cường độ sáng. Do đó, bằng cách tính toán gradient, ta có thể xác định vị trí của các cạnh này.
- Phân tích hình ảnh: Gradient cung cấp thông tin về cấu trúc và chi tiết của hình ảnh, từ đó hỗ trợ cho các nhiệm vụ như phân đoạn, nhận dạng vật thể, và theo dõi chuyển động.

Edge (biên) trong xử lý ảnh là đường bao quanh các vùng có tính chất khác nhau trong hình ảnh. Các cạnh thường tương ứng với các thay đổi đột ngột về màu sắc, cường độ, hoặc kết cấu.

Mối quan hệ giữa Gradient và Edge:

- Gradient xác định Edge: Các điểm có giá trị gradient lớn thường nằm trên các cạnh của hình ảnh.
- Edge là tập hợp các điểm gradient lớn: Một cạnh có thể được định nghĩa là một tập hợp các điểm liền kề có giá trị gradient lớn.

Các toán tử tính gradient phổ biến:

- Sobel: Toán tử Sobel là một trong những toán tử phổ biến nhất để tính gradient. Nó sử dụng hai kernel (kernel ngang và kernel dọc) để tính đạo hàm theo phương ngang và phương dọc.
- Prewitt: Tương tự như Sobel, toán tử Prewitt cũng sử dụng hai kernel để tính gradient. Tuy nhiên, các hệ số trong kernel Prewitt đơn giản hơn so với Sobel.
- Laplacian: Toán tử Laplacian tính toán Laplacian của hình ảnh, tức là tổng của đạo hàm bậc hai theo phương ngang và phương dọc. Laplacian thường được sử dụng để phát hiện các cạnh và các điểm cực trị trong hình ảnh.

Ứng dụng của Gradient và Edge:

- Phát hiện cạnh: Đây là ứng dụng cơ bản nhất của gradient và edge. Các cạnh phát hiện được có thể được sử dụng để phân đoạn hình ảnh, trích xuất các đặc trưng hình dạng, hoặc theo dõi chuyển động.
- Phân đoạn hình ảnh: Bằng cách sử dụng thông tin về gradient, ta có thể phân chia hình ảnh thành các vùng đồng nhất dựa trên các cạnh.
- Nhận dạng vật thể: Các cạnh là các đặc trưng quan trọng để nhận dạng các vật thể trong hình ảnh.

Theo dõi chuyển động: Bằng cách so sánh các cạnh trong các khung hình liên tiếp, ta có thể theo dõi chuyển động của các vật thể.

Bài thực hành chương 2

Đọc một hình ảnh bất kỳ từ máy tính và thực hiện các yêu cầu sau:

I> Toán Tử Điểm Ảnh

1. Thay đổi độ sáng: Tăng hoặc giảm độ sáng của toàn bộ hình ảnh bằng cách cộng hoặc trừ một giá trị cố định cho mỗi điểm ảnh.
2. Thay đổi độ tương phản: Tăng hoặc giảm độ tương phản bằng cách nhân mỗi điểm ảnh với một hằng số.
3. Biến đổi âm bản: Đảo ngược các giá trị điểm ảnh, tạo ra ảnh âm bản.
4. Cắt ngưỡng: Tạo ảnh nhị phân bằng cách so sánh giá trị mỗi điểm ảnh với một ngưỡng nhất định.

II> Lọc Tuyến Tính

1. Lọc trung bình: Làm mờ ảnh bằng cách tính giá trị trung bình của các điểm ảnh trong một vùng lân cận.
2. Lọc Gaussian: Làm mờ ảnh một cách tự nhiên hơn bằng cách sử dụng kernel Gaussian.
3. Làm sắc nét: Tăng cường các cạnh trong ảnh bằng cách sử dụng các kernel thích hợp.

III> Bài tập nâng cao

1. Phát hiện cạnh: Sử dụng các kernel Sobel, Prewitt để phát hiện các cạnh trong ảnh.
2. Tự thiết kế kernel: Tạo các kernel tùy chỉnh để đạt được các hiệu ứng khác nhau.
3. So sánh các loại lọc: So sánh hiệu quả của các loại lọc khác nhau trên cùng một hình ảnh.
4. Áp dụng lọc phi tuyến tính: Tìm hiểu về các loại lọc không tuyến tính như median filter, bilateral filter.

Q & A